

Angelo Reade and Connor Zynda

Professor Pincioli

RBE 4701

22 September 2026

Bombberman Agent

[Introduction]

The project involved coding an AI agent using the Python language capable of playing five different variations of a basic version of the game Bombberman. The premise of the game was to navigate through a maze to reach a target exit portal at the other end of the map. For this project, the orientation of walls on the map and location of the exit portal stayed consistent through all five variations. What changed was the presence of two different types of monsters on the map: a “stupid” monster who would move randomly around the map with no real concept of anything and a “self-preserving” monster that was capable of walking away from explosions and seeking the character once it was within a certain proximity. The first variant of the game contained an empty map with no monsters while variant two added a stupid monster to the map. Variant three replaced that stupid monster with one of the smarter self-preserving monsters and then variant four increased the aggressiveness (detection range) of that self-preserving monster. Variant five employed both a stupid monster and self-preserving monster on the map simultaneously. To take the optimal path to the portal exit cell while avoiding the monsters along the way, the group employed two main search algorithms to move the character autonomously across the map. While not in close proximity to any monsters, the character navigated to the exit cell using the Breadth First Search (BFS) algorithm. For variants two through five, the character employed BFS until it came into close proximity with a monster where it employed the Minimax algorithm. Because we couldn’t reliably check monster types in a copy of the world, we used depth-limited Minimax to account for smarter monster behavior without assuming the worst at every step. Results from testing show that the search algorithms written for navigation were generally very effective and succeeded well over the 50% success threshold.

[State Machine]

The structure chosen to complete the project was a state machine organized into three states, state entry functions, update functions, and checker-handler pairs. Initially when the program was run, the character began in a START state running two checker functions. The first checker function checked whether or not to put the monster into the SAFE_NAVIGATION state. If the map contained no monsters,

the checker would return true, switching the character to the `SAFE_NAVIGATION` state. If the character's next proposed move was within a 2 cell proximity of a monster, the checker would return false. A simultaneous checker checked whether a monster was in proximity. If the map contained no monsters, the function would automatically return false, but if the character's next move placed it within a 2 cell radius of a monster, the state machine would transfer to the `MONSTER_IN_PROXIMITY` state. If one of the checker functions returned true, its corresponding handler function would first run the state entry function contained within it, and then the update function. For example, if the `check_safe_navigation()` function returned true, its corresponding handler `Handle_Safe_Navigation()` would run the state entry function `Enter_Safe_Navigation()`, switching the state to `SAFE_NAVIGATION`. Within the update function, if statements would check whether the state machine was currently in one of the two main states. If the state machine was currently in the `SAFE_NAVIGATION` state, it would call the character to follow a BFS path to the exit cell. If the state machine was currently in the `MONSTER_IN_PROXIMITY` state, it would call the minimax evaluation function to evade the monster in proximity.

[BFS]

In order to utilize the BFS algorithm to find the optimal path in the `SAFE_NAVIGATION` state, the group created a `find_neighbors()` helper function. The `find_neighbors()` function returned a list of all legal character moves for any given cell on the map. It accounted for moves that would take the character beyond the limits of the map. The BFS algorithm took a starting node, S, and a target node, T, as inputs. It initialized a queue with the starting node and the path used to reach it. It maintained a set of visited nodes to ensure the algorithm did not process the same node multiple times. While the queue was not empty, BFS popped the first node from the queue and checked whether it was the target node. If the checked node was the target node, the algorithm returned the path to get there. If the current node was not the target, it was marked as visited and all of its neighbors along with the path to get there were added to the queue. Due to the level by level exploration nature of BFS, the first path found to the target node is the shortest and the most optimal path.

[Minimax for 1 Monster in Proximity]

Once the state machine enters the `MONSTER_IN_PROXIMITY` state, the functionality of the state machine described above results in the `minimax_evaluation` function being called. This function operates differently based on how many monster entities are present in the world. For one monster, the function finds all legal character actions using the `find_neighbors` function. It then iterates through each valid player move and gets the distance to the target node. Using a nested for loop, it then iterates through

every legal monster action using `find_neighbors`. The chebyshev distance is then calculated between character and monster for each possible monster action to see which move would make the monster behave optimally. Given the monster's optimal move for each possible player's move, an equation was calculated to quantify the quality of each move. The equation included the terms of distance to monster and distance to target node (Figure 1). In special cases such as a character move resulting in going into the same square as the monster, the resulting value from the equation was overridden by a very low value like -1000. The player move resulting from a monster acting optimally with the highest score from the equation was returned. The function `advance_to_node` was then used to have the character move to the node selected by `minimax_evaluation_function`.

[Minimax for 2 Monsters in Proximity]

When two monster entities were present in the world, the `minimax_evaluation_function` would perform depth 2 minimax. Similarly to the depth 1 minimax, the function starts by looping through every possible character action and obtaining the chebyshev distance to the target node. However in this algorithm, there are two nested for loops that get the minimum chebyshev distance from monster to character for both monsters. These two shortest distances for each monster's best possible moves were compared to see which monster posed the most significant threat. After the most dangerous monster was identified, the algorithm performed another level of minimax. The neighbors of the most optimal first move were gathered and compared with every monster's second move. The number of valid neighbor nodes for the character after his own and the monster's second moves were deemed valid escape nodes. A similar equation to depth 1 minimax was created but this one had four terms: distance from closest monster, distance from farther monster, valid escape nodes, and distance to target node (Figure 2). In special cases such as a character move resulting in going into the same square as the monster, or there being no valid escape nodes after the next move, the resulting value from the equation was overridden by a very low value like -1000. The move with the highest resulting value from the equation was returned and acted upon with the `advance_to_node` function.

[Experimental Evaluation]

To optimize the state machine to survive all variants of the game, the group underwent multiple rounds of testing to ensure it would meet the 50% threshold. While implementing the minimax algorithm to survive the monsters in the second, third, and fourth variants, the state machine ran into trouble against the more aggressive variant four monster. To tackle that issue, the group conducted thorough testing to adjust the weights in figure 1. Weight 1 was increased while weight 2 was decreased to create an optimal reward for being as far away from a monster as possible while choosing the best option to get closer to the

target. In the depth 2 minimax implementation, the group chose to reward safer behavior to ensure survival rather than risky behavior to reach the target node. Weights were adjusted accordingly in testing to match this strategy. While this design choice led to some inefficiency, it allowed for the state machine to exceed the threshold, passing 6 out of 10 times.

To test variants three through five, the group selected 10 different random seeds to test the efficiency and survival rate of the character for each new situation. The state machine expectedly passed the second variation all ten times, passed variant three seven times, passed variant four eight times, and passed variant five six times. Averaging those numbers yields an average success rate of 77.5% across all variants.

Since both students in the group have not yet taken RBE 3002, they focused heavily on strengthening understanding of the algorithms and implementing them in code. Based on the results, the group deemed the project very successful.

[Appendix]

```
w1=1
w2=.5
node_score_pairs[action] = (w1 * min_distance_to_monster) - (w2 * distance_to_target)
if min_distance_to_monster == 0:
    node_score_pairs[action] = -1000
if min_distance_to_monster == 1:
    node_score_pairs[action] = -10
```

Figure 1. Equation for depth 1 minimax to determine best character move based on optimally playing monster.

```
w1 = 2
w2 = 0.5
w3 = 0.25
w4 = 1
node_score_pairs[action] = ((w1 * biggest_monster_threat) + (w2 * lesser_monster_threat) - (w3 * distance_to_target) + (w4 * safe_escape_count) )
# Apply safety penalties
if biggest_monster_threat == 0:
    node_score_pairs[action] = -1000
else:
    if biggest_monster_threat == 1:
        node_score_pairs[action] -= 200
    if safe_escape_count == 0:
        node_score_pairs[action] -= 500
```

Figure 2. Equation for depth 2 minimax. Taking into account both monster positions, and possible escape moves after the first move was completed.